

COOKIEGUARD

Interactive Cookie and Session Security Laboratory

Marco Ponce

Seton Hall University

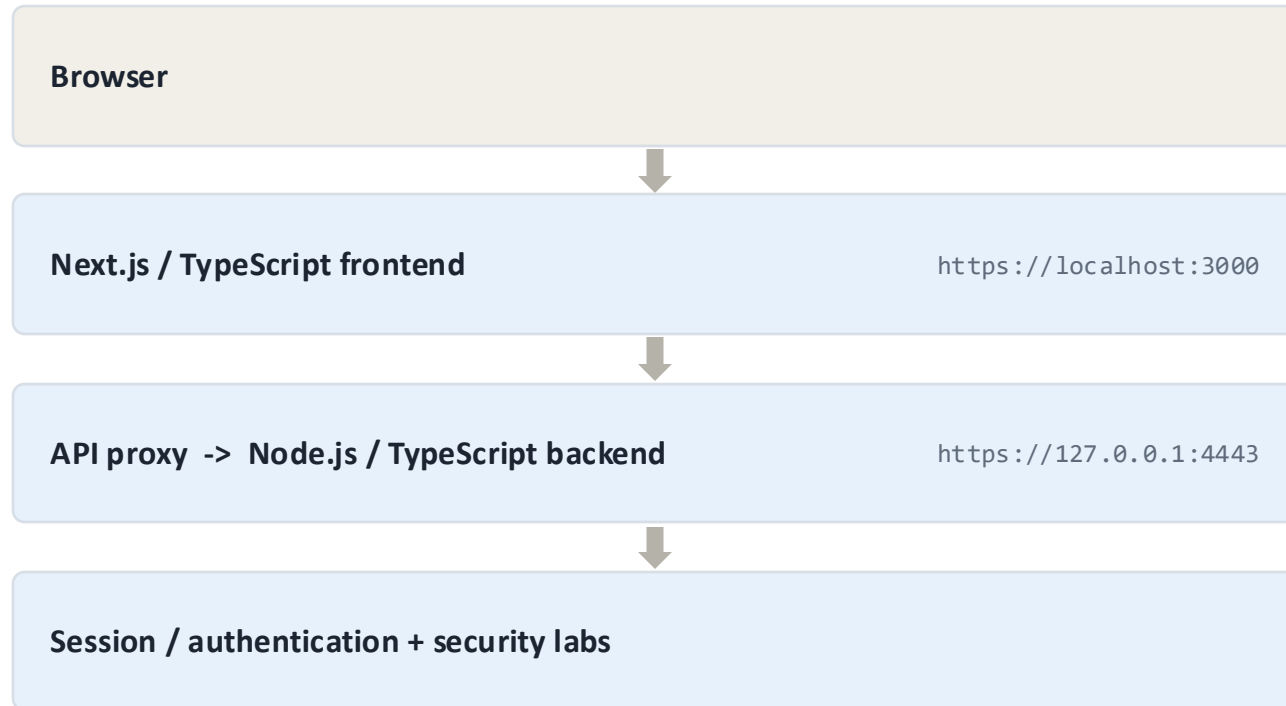
M.S. Cybersecurity

ISCI 6301: App Security & Web Technology

Problem and motivation

- Cookie and session security settings are easy to configure incorrectly and difficult to observe directly
- CookieGuard makes browser cookie and session behavior observable rather than theoretical
- It provides controlled XSS and CSRF experiments using dedicated lab cookies
- Goal: demonstrate the actual effect of common cookie security protections

Architecture



Implementation facts

- Frontend: Next.js + TypeScript
- Backend: Node.js
- Frontend runs at HTTPS localhost:3000
- Backend runs at HTTPS 127.0.0.1:4443
- Frontend proxy forwards `/api/*` to backend
- Sessions are stored in memory
- Local HTTPS uses mkcert

Cookie and session security

Secure

Cookie is sent only over HTTPS. Protects cookie transport

HttpOnly

JavaScript cannot directly read the cookie. Reduces direct cookie theft via JS

SameSite

Controls when cookies are sent with cross-site requests. Helps mitigate CSRF

cookieguard_session

- Secure = enabled
- HttpOnly = enabled
- SameSite = Lax
- Path = /

These settings reduce specific risks; they do not make the application completely secure on their own

XSS lab

Vulnerable mode

- Dedicated XSS lab cookie does NOT use HttpOnly
- Controlled JavaScript can read it using `document.cookie`

Protected mode

- Same controlled payload; HttpOnly is enabled
- JavaScript cannot directly read the cookie value

EXPERIMENT 01 · COOKIE ACCESS

HttpOnly + XSS

Use the same controlled payload twice. The only meaningful change is whether the demonstration cookie is readable by JavaScript.

This lab uses a separate demonstration cookie. The authenticated CookieGuard session is never intentionally weakened.

01 Choose the control

Set the lab cookie to vulnerable or protected mode.

Vulnerable Protected

Current mode: vulnerable

02 Run the payload

The payload attempts to read `document.cookie`.

Run XSS payload

JavaScript-visible cookies

```
cookieguard_xss_lab=lab-65c486bfc096cf0e07c2d5df4fa4f242
```

COMPARE What should change?

Vulnerable

The lab cookie lacks `HttpOnly`, so the controlled payload can read it through JavaScript.

Clear lab cookie

Protected

The lab cookie has `HttpOnly`, so the same payload cannot read that cookie.

EXPERIMENT 01 · COOKIE ACCESS

HttpOnly + XSS

Use the same controlled payload twice. The only meaningful change is whether the demonstration cookie is readable by JavaScript.

This lab uses a separate demonstration cookie. The authenticated CookieGuard session is never intentionally weakened.

01 Choose the control

Set the lab cookie to vulnerable or protected mode.

Vulnerable Protected

Current mode: protected

02 Run the payload

The payload attempts to read `document.cookie`.

Run XSS payload

JavaScript-visible cookies

COMPARE What should change?

Vulnerable

The lab cookie lacks `HttpOnly`, so the controlled payload can read it through JavaScript.

Clear lab cookie

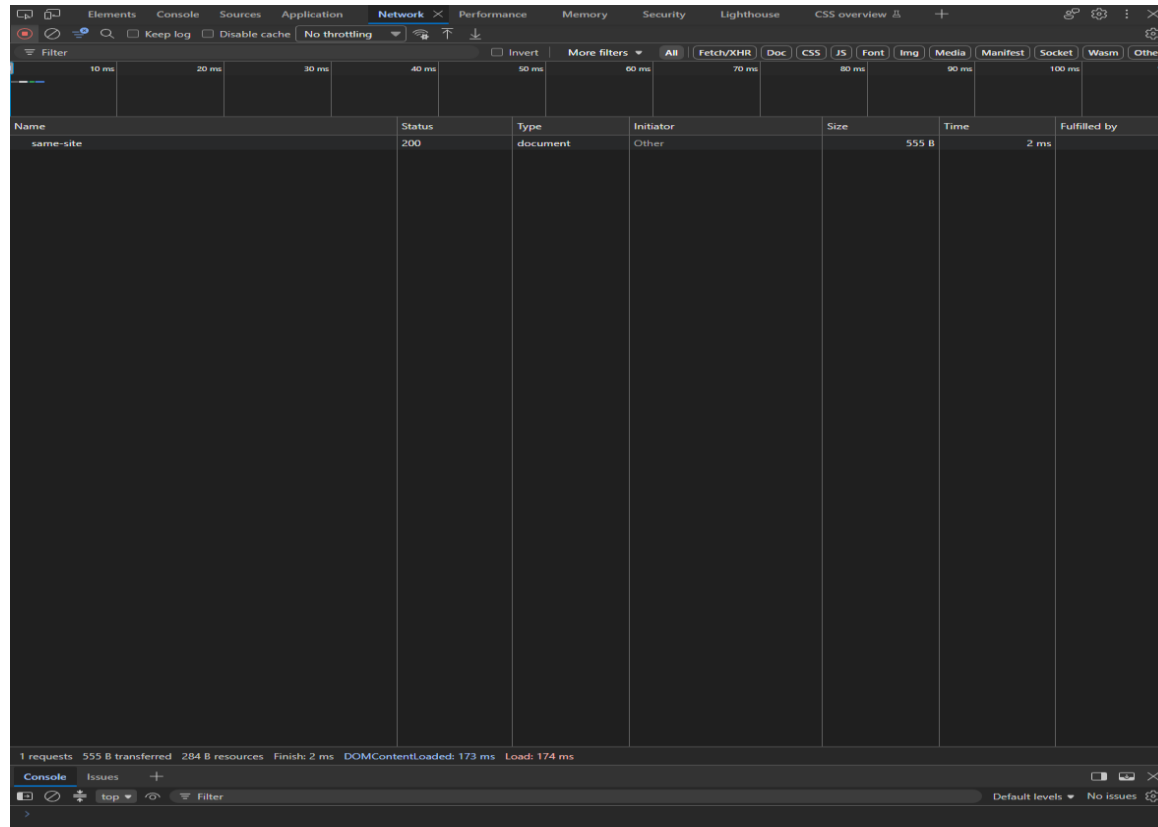
Protected

The lab cookie has `HttpOnly`, so the same payload cannot read that cookie.

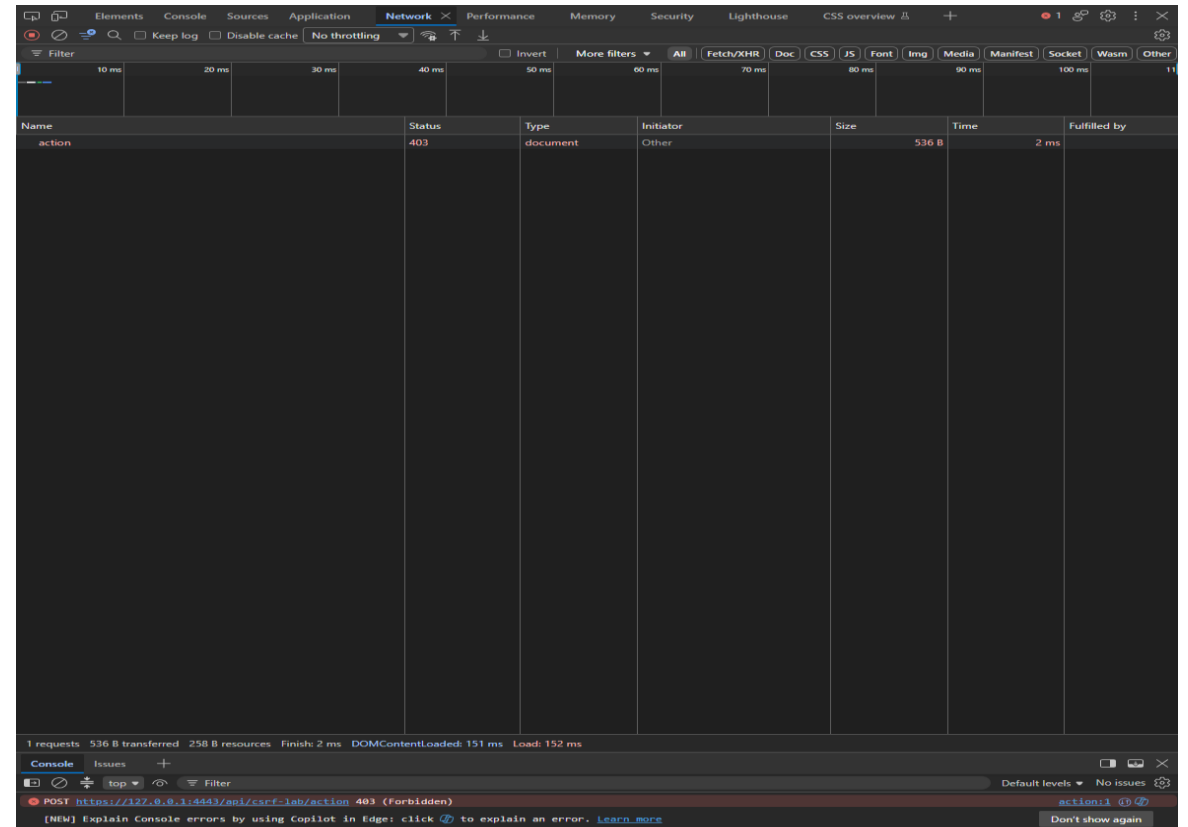
Secure remains enabled in both modes; HttpOnly does not prevent XSS itself, only direct JavaScript access to the cookie

CSRF lab

Same-site POST -> cookie attached -> HTTP 200



Cross-site POST -> cookie withheld -> HTTP 403



Authenticated session cookie uses SameSite=Lax. The dedicated CSRF lab cookie uses SameSite=Strict for a deterministic blocked case; Lax does not block every possible CSRF scenario

HTTPS, testing, and hardening

HTTPS

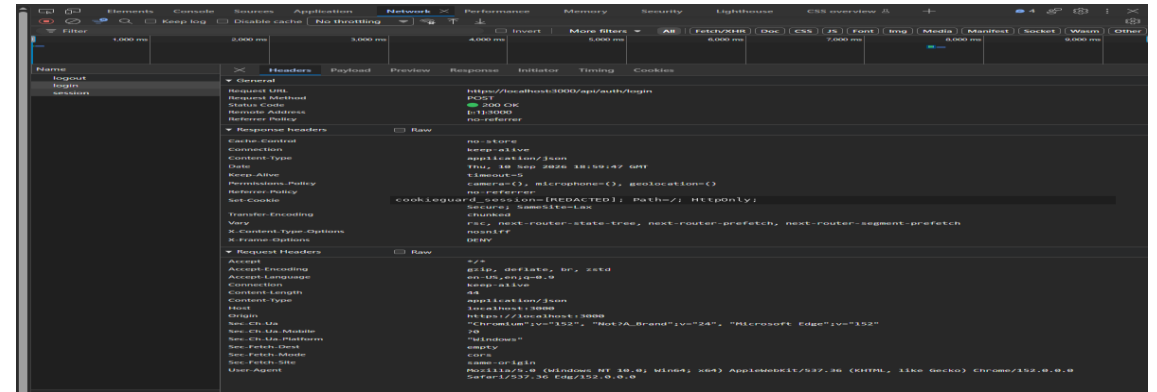
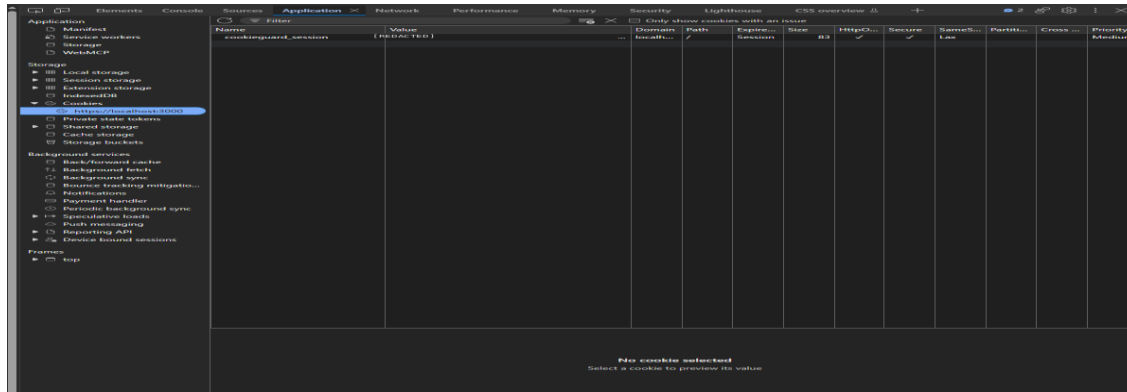
- Frontend and backend both use HTTPS
- Backend health check returned HTTP 200
- Local certificates generated with mkcert
- TLS verification was not globally disabled

Testing

- `npm run verify`
- 17 / 17 tests passed
- Backend TypeScript build passed
- Frontend production build passed
- GitHub Actions verification passed

Hardening

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- Referrer-Policy: no-referrer
- Cache-Control: no-store (where applicable)
- 64 KiB JSON body limit
- Bounded request-body accumulation



Results, limitations, and takeaway

Results

- MVP completion criteria were met
- Automated verification passed
- Manual browser testing covered the main security labs

Limitations

- Fixed demo account
- In-memory sessions
- Local mkcert certificate
- Intentional vulnerable XSS mode
- Next.js 15.5.6 should be upgraded before production use

CookieGuard makes cookie and session security behavior observable through controlled browser experiments